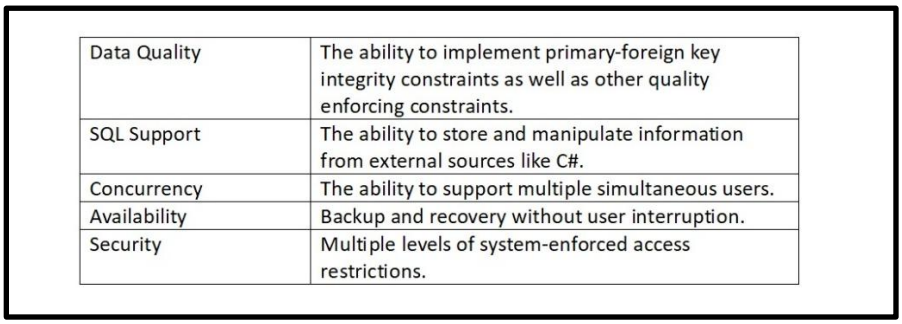
Chapter 18: Relational Database Fundamentals

In this section we are going to explore external methods for storing and manipulating your C# data. In particular, we are going to explore relational database management systems, a.k.a. RDBMSs, and XML files, i.e. the Extensible Markup Language.

We are going to start with RDBMSs, your best option for long-term data storage. Let's jump to the inside and take a look.

An RDBMS is a set of software solutions designed to store and manage information. RDBMSs provide the user with several important features. (Figure 18.1)

Figure 18.1: RDBMS Features



Data Quality	The ability to implement primary-foreign key integrity constraints as well as other quality enforcing constraints.
SQL Support	The ability to store and manipulate information from external sources like C#.
Concurrency	The ability to support multiple simultaneous users.
Availability	Backup and recovery without user interruption.
Security	Multiple levels of system-enforced access restrictions.

Each RDBMS can support multiple schemas. A schema is used to represent the information associated with one or more business functions. For example, an RDBMS might house four distinct schemas: one for the finance department, one for the sales and marketing team, one for the IT department, and one for the customer support gang. Alternatively, all this information might be housed within a singular schema. (Figures 182.A-B)

I have probably designed 10-15 large scale schemas in my career, they have always been the single integrated type. The advantage of a single integrated schema is you don't have multiple schemas to keep synchronized with data always moving between them. The big difficulty with a single schema solution is complexity. My last schema for the testing and e-learning company had over 1400 tables. That is a lot of stuff to keep track of at any given time.

Figure 18.2A: Multiple Schemas in One RDBMS

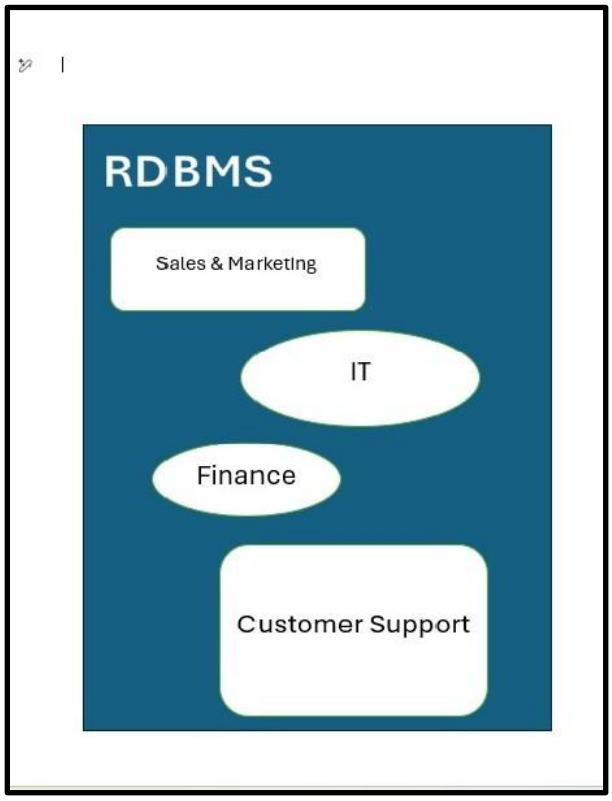
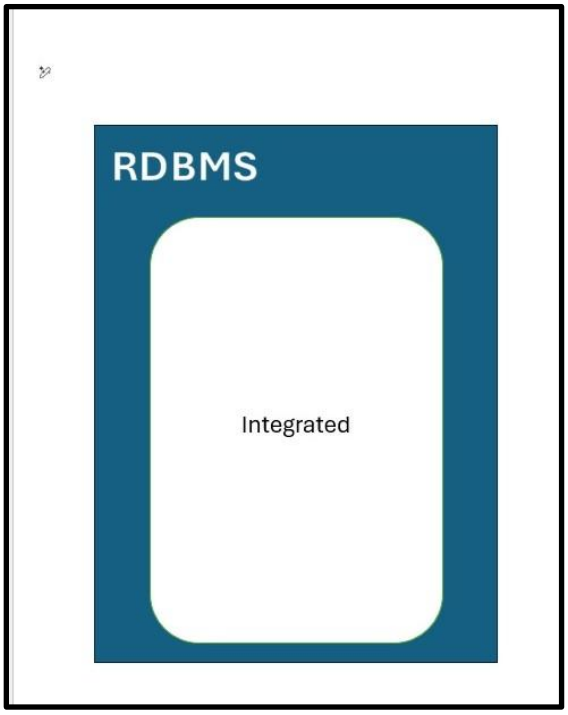


Figure 18.2B: One Large Integrated Schema



Schemas are comprised of tables and the relationships between them. Tables in turn are comprised of fields or columns each of which has a distinct data type. A row is defined as that set of fields representing a unique occurrence of the entity stored within that table. Each row is uniquely identified by a subset of the row's fields known as the primary key. If that same set of fields exists within a different table that set of fields in the secondary table is known as a foreign key. (Figure 18.3)

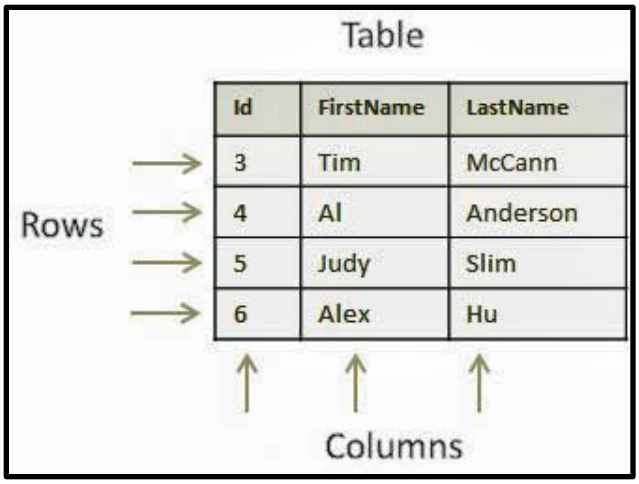
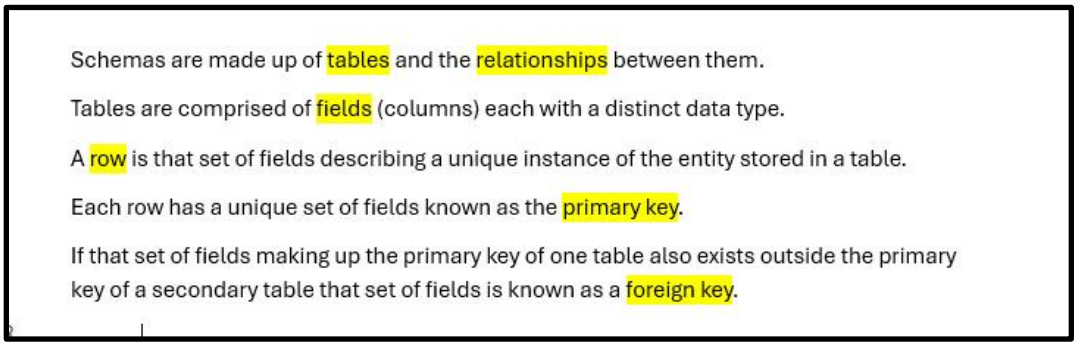


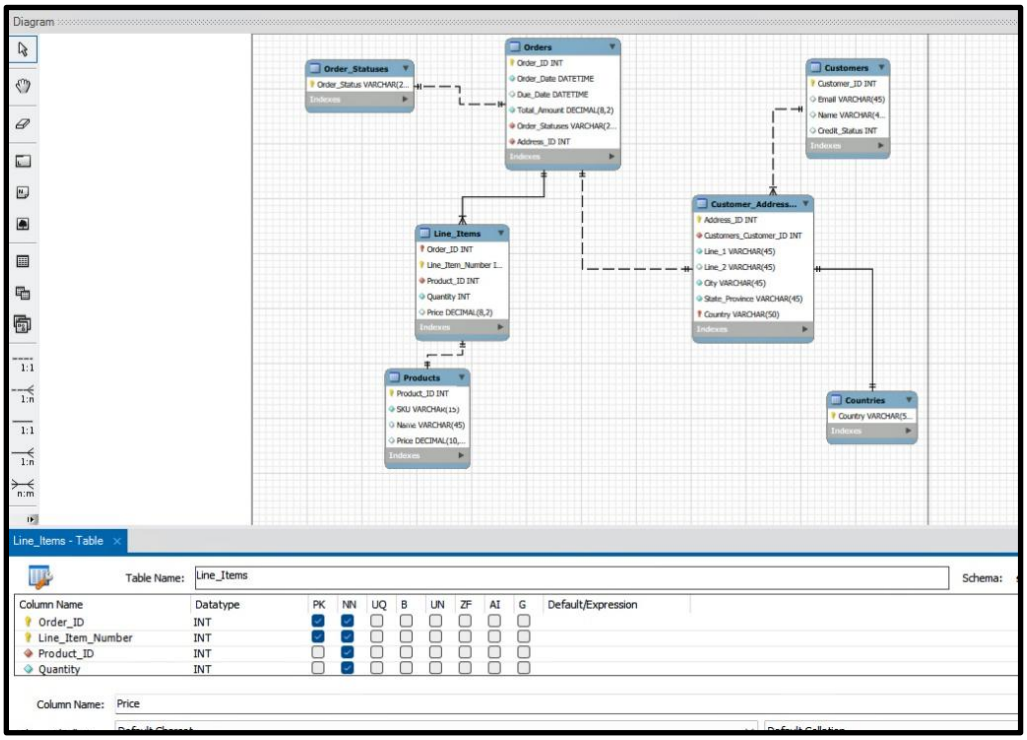
Figure 18.3: RDBMS Terminology



An example will help to clarify all this terminology. In this very simple entity-relationship diagram all the tables are represented by rectangles. (Figure 18.4) The relationships between those tables are represented by lines. Inside each table is a list of the fields making up a row, or instance of the entity stored within that table. The data type of each field is indicated next to the field name. If the field is an element of the primary key it is labeled with a yellow key icon. If it is part of a foreign key it is labeled with a red diamond.

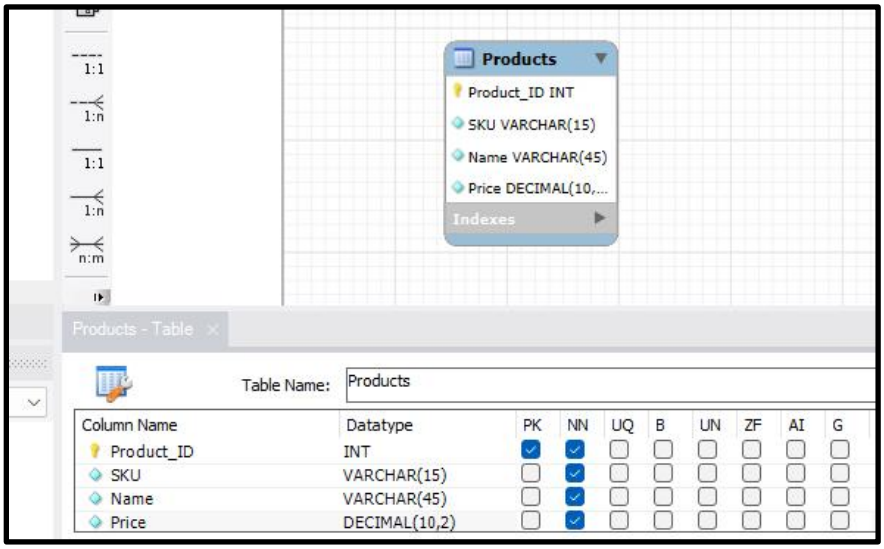
By the way, I created this entity-relationship diagram using Microsoft's SQL Server Management Studio. I strongly recommend you recreate it using your copy of this software. It is a good tool with which to get aquainted.

Figure 18.4 Simple Entity-Relationship Diagram



Let's zoom in on the Products table to put some of this terminology into action. Please note that the Products table has four columns or fields. Each has a distinct data type and none of the fields are nullable (NN). The Product_ID field is of type integer and is the primary key (PK) of the table. (Figure 18.5)

Figure 18.5: Products Table



When we write out rows in this table we do so as follows:

Products (Product_ID, SKU, Name, Price)

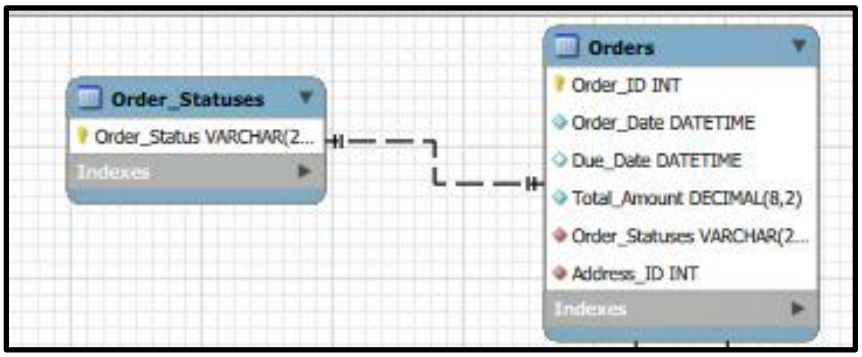
Products (0, 'A15623', 'Bob''s Big Ball', 15.99)
Products (1, 'B987654', 'Tofu', 4.23)
Products (2, 'C456123', 'Eggs', 6.87)
Products (3, 'D51948', 'Peaches', 4.25)

So, what did we learn about our Products table?

- First, it was named in the plural. Either singular or plural is acceptable. I always used the plural form but there is really no standard.
- Next, any given row must conform to the specified field definitions. In our case that means that each row has to have a Product_ID, SKU, Name and Price field. None of these fields are nullable.
- Each row must be uniquely identified via a primary key. In the case of our Products table the Product_ID uniquely identifies each row, so it is the primary key. When written out as shown here, the primary key field(s) are always underlined.
- In other tables some of the fields may be omitted when written out. When omitted from these tables these fields are indicated via the keyword null. When the table is defined by the database designer, he/she will indicate which, if any, of the fields are nullable.
- String literals are delimited in SQL via single quotes. To embed a single quote within a string double the single quotes, e.g. 'Bob''s Big Ball'.

Next let's take a look at the Orders and Order_Statuses tables. The dashed line between these two tables indicates a relationship between the tables. In this case the line has no crow's feet on either end so the relationship is known as a 1:1 relationship. Think about the Order_Statuses table as a listing of all the valid order statuses, e.g. In-Progress, Filled, Out of Stock, etc. The 1:1 relationship between these tables means that each instance of an Order has exactly one valid status as defined by the entries in the Order_Statuses table. (Figure 18.6)

Figure 18.6: Orders to Order_Statuses 1:1



The primary key of the Order_Statuses table is the Order_Status field. Because that same field exists in the Orders table and is not part of the primary key of that table it is known as a foreign key. Foreign keys are used to enforce data quality in a schema. You cannot delete a row in the Order_Statuses table if it has been used in any row in the Orders table. Similarly, you cannot enter any Order_Status value in the Orders table that does not exist in the Order_Statuses table.

By the way, the Address_ID field in the Orders table is also a foreign key back to the Customer_Addresses table. The same data quality rules would apply between those two tables.

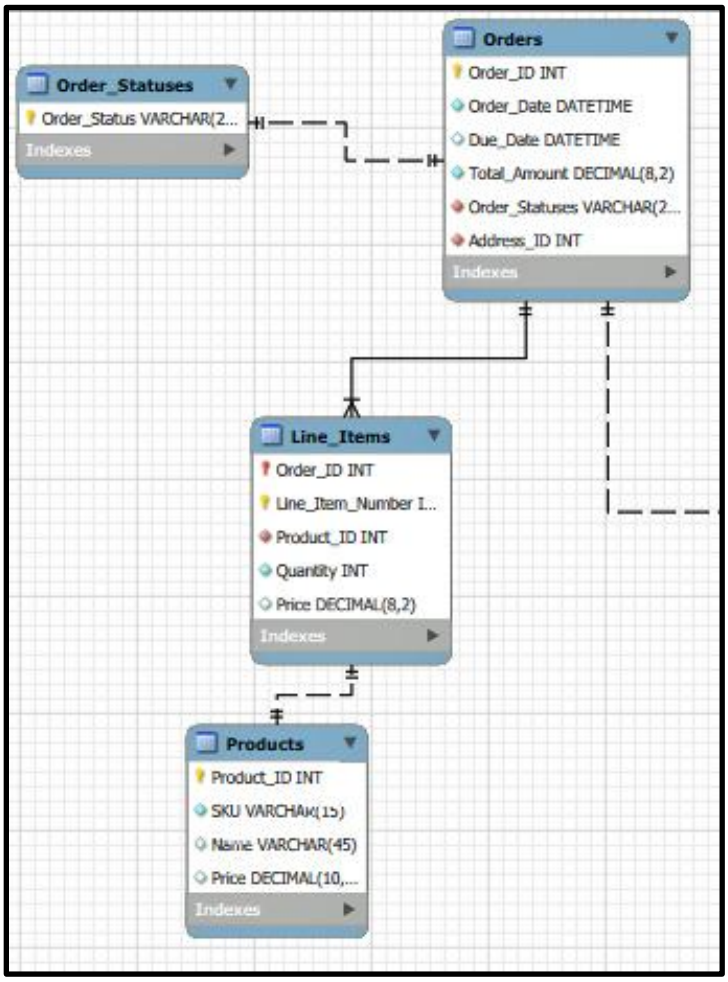
Orders (Order_ID, Order_Date, Due_Date, Total_Amount, Order_Status*, Adress_ID*)
Order_Statuses (Order_Status)

Order_Statuses (‘Unfilled’)
Order_Statuses (‘In-Progress’)
Order_Statuses (‘Filled’)
Order_Statuses (‘Abandoned’)

Orders (1, ’12-12-2025’,’12-14-2025’,102.65,’Unfilled’,1)
Orders (2, ’12-15-2025’,’12-15-2025’,1002.65,’In-Progress’,2)
Orders (3, ’10-01-2025’,’10-10-2025’,2.65,’Abandoned’,2)

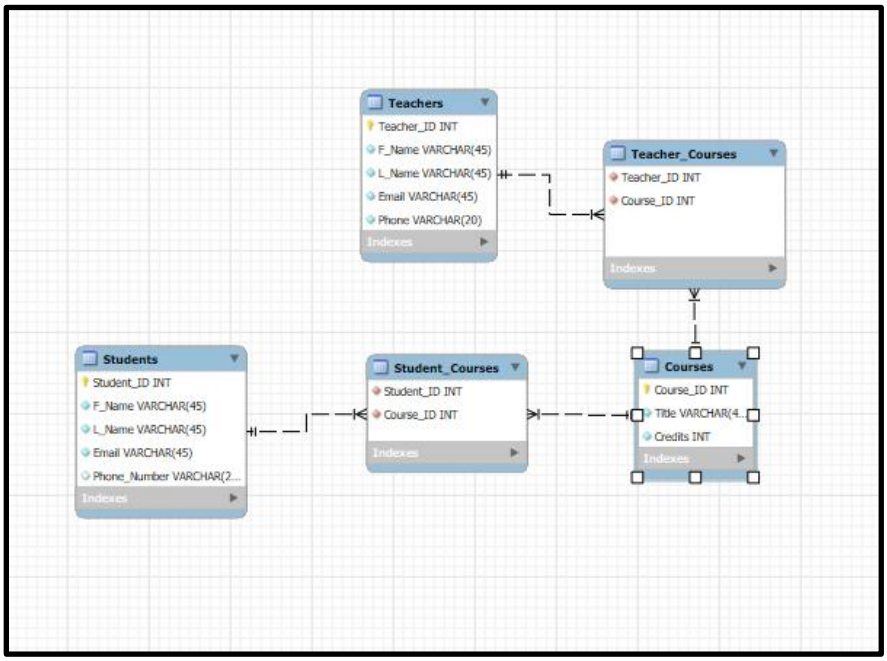
The relationship between the Orders and Line_Items tables has a crow’s foot on the Line_Items end. This infers a 1:M relationship between these two tables. Each row in the Orders table may be associated with zero, one or many instances in the Line_Items table. Referential integrity, yes that is what you call the data quality rules enforced by primary to foreign key relationships, hold between these two tables. Please also note how there is a 1:1 relationship between the Line_Items table and the Products table. This ensures that all products referenced in the Line_Items table are valid products. (Figure 18.7)

Figure 18.7: Orders to Line_Items 1:M



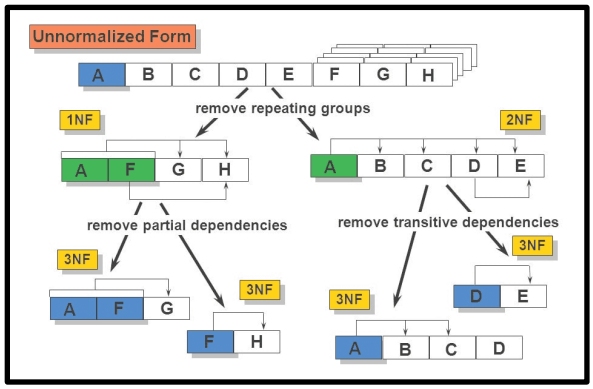
A valid question at this point would be is there such a thing as a M:N relationship. The answer is yes, but it is not realized as you might expect. Take a look at this ER Diagram. (Figure 18.8) It is a classic M:N example.

Figure 18.8: Many:Many (M:N) Relationship



This ER depicts three real-world entities: Students, Courses and Teachers. Let's assume you speak with the subject matter experts and they tell you that any teacher may teach zero, one or many courses and that any student may enroll in zero, one or many courses. To realize these two M:N relationships we need two intermediary tables each with a compound primary key made up of the associated real-world entity primary keys. In our case the Teachers:Courses M:N relationship is realized via the Teacher_Courses table which has a compound primary key made up of the Teacher_ID and Course_ID fields. The Students:Courses M:N relationship is realized via the Student_Courses table and has a primary key made up of the Student_ID and Course_ID fields.

This wraps up our discussion of relational database theory. We could go into the normalization process, the process by which you determine which fields go into which tables, but for that you will need to take my PL/SQL class as that process is well beyond the scope of this course.



Cool, in our next lesson we are going to explore the wonderful world of SQL, the Structured Query Language. The language by which you manipulate data within an RDBMS.